

dem ser atribuídas na declaração ou com um construtor estático.¹¹ Então, se um programa precisa de um objeto de valor constante, cujo valor é o mesmo em cada uso de um programa, uma constante **const** é usada. Entretanto, se um programa precisa de um objeto de valor constante, cujo valor é determinado apenas quando o objeto é criado e pode ser diferente para execuções diversas do programa, uma constante **readonly** é usada.

A discussão sobre valores vinculados a constantes nomeadas naturalmente leva ao assunto da inicialização, porque vincular um valor a uma constante nomeada é o mesmo processo, exceto por ser permanente.

Em muitos casos, é conveniente para as variáveis ter valores antes de o código do programa ou subprograma no qual elas são declaradas começar a executar. A vinculação de uma variável a um valor no momento em que ela é vinculada ao armazenamento é chamada de **inicialização**. Se a variável é estaticamente vinculada ao armazenamento, a vinculação e a inicialização ocorrem antes do tempo de execução. Nesses casos, o valor inicial deve ser especificado como um literal, ou como uma expressão cujos operandos não literais sejam constantes nomeadas já definidas. Se a vinculação ao armazenamento for dinâmica, a inicialização também é dinâmica e os valores iniciais podem ser quaisquer expressões.

Na maioria das linguagens, a inicialização é especificada na declaração que cria a variável. Por exemplo, em C++, poderíamos ter

```
int sum = 0;
int* ptrSum = &sum;
char name[] = "George Washington Carver";
```

RESUMO

A distinção entre maiúsculas e minúsculas e o uso de sublinhados são questões de projeto para nomes.

As variáveis podem ser caracterizadas por seis atributos: nome, endereço, valor, tipo, tempo de vida e escopo.

Apelidos são duas ou mais variáveis vinculadas ao mesmo endereço de armazenamento. Eles são considerados prejudiciais à confiabilidade, mas são difíceis de serem eliminados completamente de uma linguagem.

A vinculação é a associação de atributos a entidades de programa. O conhecimento dos tempos de vinculação de atributos a entidades é essencial para se entender a semântica das linguagens de programação. A vinculação pode ser estática ou dinâmica. Declarações, tanto explícitas quanto implícitas, fornecem uma forma de especificar a vinculação estática de variáveis a tipos. Em geral, a vinculação dinâmica permite maior flexibilidade, à custa de legibilidade, eficiência e confiabilidade.

Variáveis escalares podem ser separadas em quatro categorias, considerando seus tempos de vida: estáticas, dinâmicas da pilha, dinâmicas do monte explícitas e dinâmicas do monte implícitas.

¹¹Em C#, os construtores estáticos são executados em algum momento indeterminado, antes que a classe seja instanciada.

O escopo estático é um recurso central de ALGOL 60 e de alguns de seus descendentes. Ele fornece um método simples, confiável e eficiente de permitir visibilidade a variáveis não locais em subprogramas. O escopo dinâmico fornece mais flexibilidade do que o escopo estático, mas à custa de legibilidade, confiabilidade e eficiência.

A maioria das linguagens funcionais permite que o usuário crie escopos locais com construções `let`, as quais limitam o escopo dos nomes definidos.

O ambiente de referenciamento de uma sentença é a coleção de todas as variáveis visíveis para aquela sentença.

Constantes nomeadas são simplesmente variáveis vinculadas a valores apenas uma vez.

QUESTÕES DE REVISÃO

1. Quais são as questões de projeto para nomes?
2. Qual é o perigo em potencial dos nomes sensíveis à distinção maiúsculas/minúsculas?
3. O que é um apelido?
4. Que categoria de variáveis de referência em C++ é sempre composta de apelidos?
5. O que é o lado *esquerdo* de uma variável? O que é o lado *direito*?
6. Defina *vinculação* e *tempo de vinculação*.
7. Após o projeto e a implementação de uma linguagem, quais são os quatro tipos de vinculações que podem ocorrer em um programa?
8. Defina *vinculação estática* e *vinculação dinâmica*.
9. Quais são as vantagens e desvantagens das declarações implícitas?
10. Quais são as vantagens e desvantagens da vinculação de tipos dinâmica?
11. Defina *variáveis estáticas*, *dinâmicas da pilha*, *dinâmicas do monte explícitas* e *dinâmicas do monte implícitas*. Quais são suas vantagens e desvantagens?
12. Defina *tempo de vida*, *escopo*, *escopo estático* e *escopo dinâmico*.
13. Como a referência a uma variável não local em um programa com escopo estático está conectada à sua definição?
14. Qual é o problema geral do escopo estático?
15. O que é o ambiente de referenciamento de uma sentença?
16. O que é um ancestral estático de um subprograma? O que é um ancestral dinâmico de um subprograma?
17. O que é um bloco?
18. Qual é a finalidade das construções `let` em linguagens funcionais?
19. Qual é a diferença entre os nomes definidos em uma construção `let` de ML e as variáveis declaradas em um bloco de C?

20. Descreva o encapsulamento de um `let` de F#, dentro de uma função e fora de todas as funções.
21. Quais são as vantagens e as desvantagens do escopo dinâmico?
22. Quais são as vantagens das constantes nomeadas?

PROBLEMAS

1. Qual das seguintes formas de identificador é mais legível? Argumente.

```
SumOfSales
sum_of_sales
SUMOFSALES
```

2. Algumas linguagens de programação não têm tipos. Quais são as vantagens e desvantagens óbvias da ausência de tipos em uma linguagem?
3. Escreva uma sentença de atribuição simples com um operador aritmético em alguma linguagem que você conheça. Para cada componente da sentença, liste as vinculações necessárias para determinar a semântica quando a sentença é executada. Para cada vinculação, indique o tempo de vinculação usado para a linguagem.
4. A vinculação de tipos dinâmica está muito relacionada às variáveis dinâmicas do monte. Explique esse relacionamento.
5. Descreva uma situação na qual uma variável sensível ao histórico em um subprograma é útil.
6. Considere o seguinte esqueleto de programa em JavaScript:

```
// O programa principal
var x;
function sub1() {
    var x;
    function sub2() {
        . . .
    }
}
function sub3() {
    . . .
}
```

Assuma que a execução desse programa é na seguinte ordem de unidades:

```
main chama sub1
sub1 chama sub2
sub2 chama sub3
```

- a. Supondo escopo estático, a seguir, qual declaração de `x` é a correta para uma referência a `x`?
 - i. `sub1`
 - ii. `sub2`
 - iii. `sub3`
 - b. Repita o exercício a, mas suponha escopo dinâmico.
7. Suponha que o seguinte programa em JavaScript foi interpretado por meio de regras de escopo estático. Que valor de `x` é impresso na função `sub1`? Sob regras de escopo dinâmico, qual valor de `x` é impresso na função `sub1`?

```
var x;
function sub1() {
  document.write("x = " + x + "");
}
function sub2() {
  var x;
  x = 10;
  sub1();
}
x = 5;
sub2();
```

8. Considere o seguinte programa em JavaScript:

```
var x, y, z;
function sub1() {
  var a, y, z;
  function sub2() {
    var a, b, z;
    . . .
  }
  . . .
}
function sub3() {
  var a, x, w;
  . . .
}
```

Liste todas as variáveis, com as unidades de programa onde elas estão declaradas, visíveis nos corpos de `sub1`, `sub2` e `sub3`, supondo que é usado escopo estático.

9. Considere o seguinte programa em Python:

```
x = 1;
y = 3;
z = 5;
def sub1():
  a = 7;
  y = 9;
  z = 11;
```

```
. . .
def sub2():
    global x;
    a = 13;
    x = 15;
    w = 17;
    . . .
def sub3():
    nonlocal a;
    a = 19;
    b = 21;
    z = 23;
    . . .
. . .
```

Liste todas as variáveis, com as unidades de programa onde elas estão declaradas, visíveis nos corpos de sub1, sub2 e sub3, supondo que é usado escopo estático.

10. Considere o seguinte programa em C:

```
void fun(void) {
    int a, b, c; /* definição 1 */
    . . .
    while ( . . . ) {
        int b, c, d; /*definição 2 */
        . . . <----- 1
        while ( . . . ) {
            int c, d, e; /* definição 3 */
            . . . <----- 2
        }
        . . . <----- 3
    }
    . . . <----- 4
}
```

Para cada um dos quatro pontos marcados nessa função, liste cada variável visível, com o número da sentença de definição que a define.

11. Considere o seguinte esqueleto de programa em C:

```
void fun1(void); /* protótipo */
void fun2(void); /* protótipo */
void fun3(void); /* protótipo */
void main() {
    int a, b, c;
    . . .
}
void fun1(void) {
    int b, c, d;
    . . .
}
void fun2(void) {
```

```

    int c, d, e;
    . . .
}
void fun3(void) {
    int d, e, f;
    . . .
}

```

Dadas as seguintes sequências de chamadas e supondo que o escopo dinâmico é usado, que variáveis são visíveis durante a execução da última função chamada? Inclua com cada variável visível o nome da função na qual foi definida.

- main chama fun1; fun1 chama fun2; fun2 chama fun3.
- main chama fun1; fun1 chama fun3.
- main chama fun2; fun2 chama fun3; fun3 chama fun1.
- main chama fun3; fun3 chama fun1.
- main chama fun1; fun1 chama fun3; fun3 chama fun2.
- main chama fun3; fun3 chama fun2; fun2 chama fun1.

12. Considere o programa a seguir, escrito em sintaxe do tipo JavaScript:

```

// programa principal
var x, y, z;

function sub1() {
    var a, y, z;
    . . .
}
function sub2() {
    var a, b, z;
    . . .
}
function sub3() {
    var a, x, w;
    . . .
}

```

Dadas as seguintes sequências de chamadas e supondo que é usado escopo dinâmico, quais variáveis são visíveis durante a execução do último subprograma ativado? Inclua com cada variável visível o nome da função na qual ela foi declarada.

- main chama sub1; sub1 chama sub2; sub2 chama sub3.
- main chama sub1; sub1 chama sub3.
- main chama sub2; sub2 chama sub3; sub3 chama sub1.
- main chama sub3; sub3 chama sub1.
- main chama sub1; sub1 chama sub3; sub3 chama sub2.
- main chama sub3; sub3 chama sub2; sub2 chama sub1.